

TraceLab

Visual Runtime Platform for Learning Programming

October 4, 2026

Contents

1	Executive Summary	3
2	Problem Statement	3
3	Vision	4
4	Goals	4
4.1	Primary Goals	4
4.2	Secondary Goals	4
5	Target Audience	4
5.1	Primary Audience	4
5.2	Secondary Audience	4
5.3	Tertiary Audience	5
6	Core Product Concept	5
7	Key Features	5
7.1	Visual Runtime Engine	5
7.2	Line-by-Line Execution	5
7.3	Memory Visualization	6
7.4	Expression Visualization	6
7.5	Conditional Visualization	6
7.6	Loop Visualization	6
7.7	Array Visualization	7
7.8	Function Call Visualization	7
7.9	Recursion Visualization	7
7.10	Algorithm Visualization	7
7.11	Debug Mode	7
7.12	Why Did This Happen?	8
7.13	Predict Next Step	8
8	Learning Modes	8
8.1	Playground Mode	8
8.2	Guided Learning Mode	8
8.3	DSA Lab	8
8.4	LeetCode Visualizer	9

9	Supported Languages	9
9.1	Phase 1	9
9.2	Phase 2	9
9.3	Phase 3	9
9.4	Phase 4	9
10	System Architecture	9
11	Execution Event Model	10
12	Technology Stack	10
12.1	Frontend	10
12.2	Backend	11
12.3	Parser Layer	11
12.4	Database	11
12.5	Authentication	11
12.6	Deployment	11
13	User Interface Layout	11
14	Gamification	12
15	MVP Scope	12
16	Success Metrics	12
16.1	User Metrics	12
16.2	Learning Metrics	13
17	Future Roadmap	13
17.1	Version 2	13
17.2	Version 3	13
17.3	Version 4	13
17.4	Version 5	13
18	Final Positioning	13

1 Executive Summary

TraceLab is an interactive programming learning platform that helps beginners understand how code executes internally through real-time visualizations.

Instead of only showing the final output, TraceLab visualizes:

- Variable creation
- Memory allocation
- Expression evaluation
- Conditional execution
- Loops
- Arrays
- Functions
- Recursion
- Data structures
- Algorithms

The platform acts as a visual debugger specifically designed for education.

Users write code in an editor and watch every execution step happen visually.

2 Problem Statement

Most beginners learn programming by reading code and observing output.

Example:

```
int a = 2;
int b = 3;
int sum = a + b;
```

Output:

5

However, beginners never see:

- How variables are created
- Where values are stored
- How expressions are evaluated
- Why conditions are true or false
- How loops iterate
- What changes after each line executes

This creates gaps in understanding and makes debugging and algorithm learning difficult.

3 Vision

Create the most intuitive visual programming learning platform that allows users to understand code execution step-by-step.

Core Question:

“What exactly is happening inside my program right now?”

4 Goals

4.1 Primary Goals

- Help beginners understand code execution
- Improve debugging skills
- Accelerate learning of programming fundamentals
- Simplify DSA concepts
- Improve LeetCode problem-solving ability

4.2 Secondary Goals

- Reduce programming anxiety
- Increase learning engagement
- Improve retention
- Support classroom teaching

5 Target Audience

5.1 Primary Audience

Beginner programmers:

- University students
- School students
- Bootcamp learners
- Self-taught developers

5.2 Secondary Audience

Intermediate developers who want to:

- Learn DSA
- Understand recursion
- Visualize algorithms

5.3 Tertiary Audience

Educators and instructors using visual demonstrations during lectures and labs.

6 Core Product Concept

User writes code.

Example:

```
int a = 2;
int b = 3;
int sum = a + b;
```

TraceLab parses the code and generates execution events.

Visualization:

```
Create variable a
Store value 2
```

```
Create variable b
Store value 3
```

```
Read a
Read b
```

```
Compute 2 + 3
```

```
Create variable sum
Store value 5
```

No print statements are required.

7 Key Features

7.1 Visual Runtime Engine

Transforms code execution into visual events.

Example:

```
int x = 10;
```

Visualization:

```
+-----+
| x = 10 |
+-----+
```

7.2 Line-by-Line Execution

Controls:

- Run
- Pause

- Next Step
- Previous Step
- Restart

Current executing line is highlighted.

7.3 Memory Visualization

Variables are displayed as memory boxes.

7.4 Expression Visualization

Example:

```
int sum = a + b;
```

Visualization:

a -> 2

b -> 3

2 + 3

=

5

7.5 Conditional Visualization

Example:

```
if(a > b)
```

Visualization:

2 > 3

FALSE

Execute ELSE branch

7.6 Loop Visualization

Example:

```
for(int i=0;i<3;i++)
```

Visualization:

Iteration 1

i = 0

0 < 3

TRUE

7.7 Array Visualization

Example:

```
int arr[] = {4,2,7,1};
```

Visualization:

Index

```
0    1    2    3
```

```
+---+---+---+---+
| 4 | 2 | 7 | 1 |
+---+---+---+---+
```

7.8 Function Call Visualization

Displays call stack and parameter values.

7.9 Recursion Visualization

Shows stack growth and recursive returns.

7.10 Algorithm Visualization

Supported examples:

- Bubble Sort
- Selection Sort
- Merge Sort
- Binary Search
- BFS
- DFS
- Two Sum
- Sliding Window

7.11 Debug Mode

Identifies common logical mistakes.

Example:

```
for(int i=0;i<=n;i++)
```

System highlights potential out-of-bound issues.

7.12 Why Did This Happen?

Users can inspect any execution step.

Example:

Condition:

`a > b`

`2 > 3`

`Result = FALSE`

Therefore `ELSE` executes.

7.13 Predict Next Step

Interactive learning mode.

Users predict execution results before revealing the answer.

8 Learning Modes

8.1 Playground Mode

Free coding environment.

Users write and visualize code execution.

8.2 Guided Learning Mode

Topics include:

- Variables
- Data Types
- Operators
- Conditions
- Loops
- Functions
- Recursion
- Arrays
- Strings

8.3 DSA Lab

Interactive learning of:

- Arrays
- Linked Lists

- Stacks
- Queues
- Trees
- Graphs
- Hash Maps

8.4 LeetCode Visualizer

Paste a solution and watch the algorithm execute visually.

9 Supported Languages

9.1 Phase 1

JavaScript

9.2 Phase 2

Python

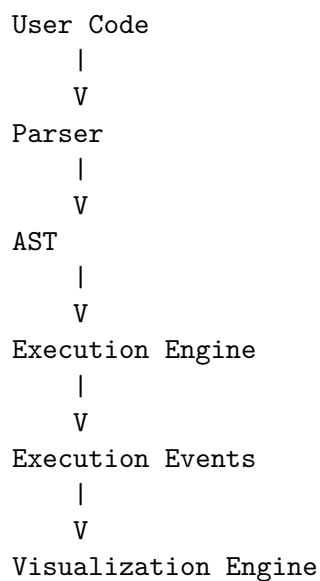
9.3 Phase 3

Java

9.4 Phase 4

C++

10 System Architecture



11 Execution Event Model

Example Input:

```
let a = 2;
let b = 3;
let sum = a + b;
```

Generated Events:

```
[
  {
    "type": "create",
    "variable": "a",
    "value": 2
  },
  {
    "type": "create",
    "variable": "b",
    "value": 3
  },
  {
    "type": "read",
    "variable": "a"
  },
  {
    "type": "read",
    "variable": "b"
  },
  {
    "type": "operation",
    "operator": "+",
    "result": 5
  },
  {
    "type": "create",
    "variable": "sum",
    "value": 5
  }
]
```

12 Technology Stack

12.1 Frontend

- Next.js 16
- TypeScript
- Tailwind CSS v4
- shadcn/ui
- Framer Motion

- Monaco Editor
- Zustand
- React Flow

12.2 Backend

- Next.js Route Handlers
- TypeScript

12.3 Parser Layer

- Babel Parser
- Acorn
- ESTree

12.4 Database

- PostgreSQL
- Prisma ORM

12.5 Authentication

- NextAuth v5
- Google Login
- GitHub Login

12.6 Deployment

- Vercel
- Neon PostgreSQL

13 User Interface Layout

+-----+ Navigation Bar +-----+		
+-----+-----+ Code Editor Visualization Panel +-----+-----+		
+-----+ Timeline / Execution Events +-----+		

14 Gamification

- XP System
- Achievements
- Streaks
- Badges
- Learning Progress
- Leaderboards

15 MVP Scope

Included:

- Variables
- Assignments
- Arithmetic Operations
- Comparisons
- If-Else
- Loops
- Arrays
- Functions
- Return Values

Excluded:

- Classes
- Multithreading
- Async Programming
- Advanced Pointers
- External Libraries

16 Success Metrics

16.1 User Metrics

- Daily Active Users
- Weekly Active Users
- Session Duration
- Lesson Completion Rate

16.2 Learning Metrics

- Debugging Accuracy
- Problem Solving Improvement
- LeetCode Success Rate

17 Future Roadmap

17.1 Version 2

- Python Support
- Object Visualization
- Class Visualization

17.2 Version 3

- Linked List Visualization
- Tree Visualization
- Graph Visualization

17.3 Version 4

- AI Tutor
- AI Explanations
- AI Code Review

17.4 Version 5

- Classroom Mode
- Instructor Dashboard
- Student Analytics

18 Final Positioning

TraceLab is not merely an online compiler, coding course, or LeetCode clone.

It is a Visual Runtime Platform that enables learners to observe code execution in real time, helping them develop a deep understanding of programming, debugging, data structures, and algorithms through interactive visualizations.